

RETRIEVAL MEANING

The word **retrieval** in Retrieval-Augmented Generation has a precise meaning: *finding and fetching external information that is relevant to a given query, before the language model generates its response*

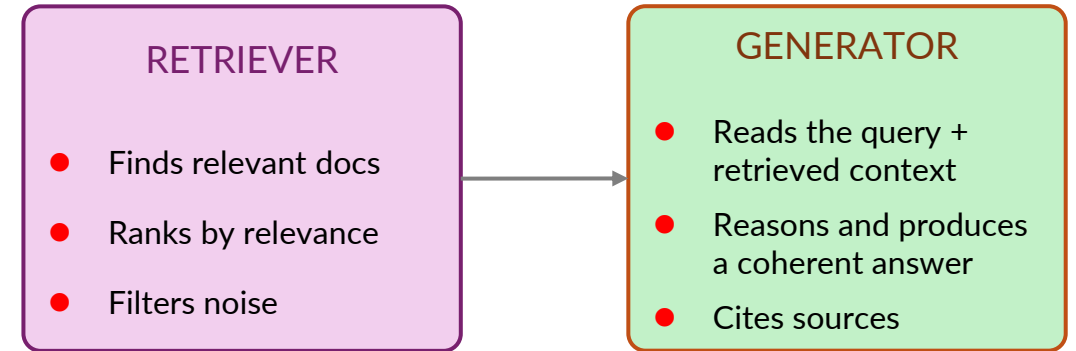
The model is handed the evidence to answer a query

	Without Retrieval	With Retrieval
Source of knowledge	Frozen model parameters; Answering from memory	Live document store; Answering with a knowledge base
Updatable?	Only by retraining	Yes, update the store anytime
Verifiable?	No source to check	Each answer cites its sources

RETRIEVAL vs GENERATION

A RAG system cleanly separates two concerns:

- The **retriever** owns the "what" part of RAG
 - It tells which documents, passages, or chunks are relevant to the user's question
 - It does not understand language deeply
 - It performs fast mathematical similarity over vectors
- The **generator** (the LLM) owns the "how" part of RAG
 - It synthesises a fluent, accurate response from the query plus the retrieved context
 - It does not search for information
 - It works only with what the retriever hands it



WHY SEPARATE RETRIEVAL-GENERATION?

It is important to be clear about the retriever's boundaries:

- It does not understand the question. It computes mathematical similarity.
- It does not generate text. It returns existing document chunks verbatim.
- It does not reason. It cannot combine information from multiple chunks.
- It does not verify facts. It ranks by relevance, not by correctness.

All reasoning, synthesis, and language generation happen in the generator.

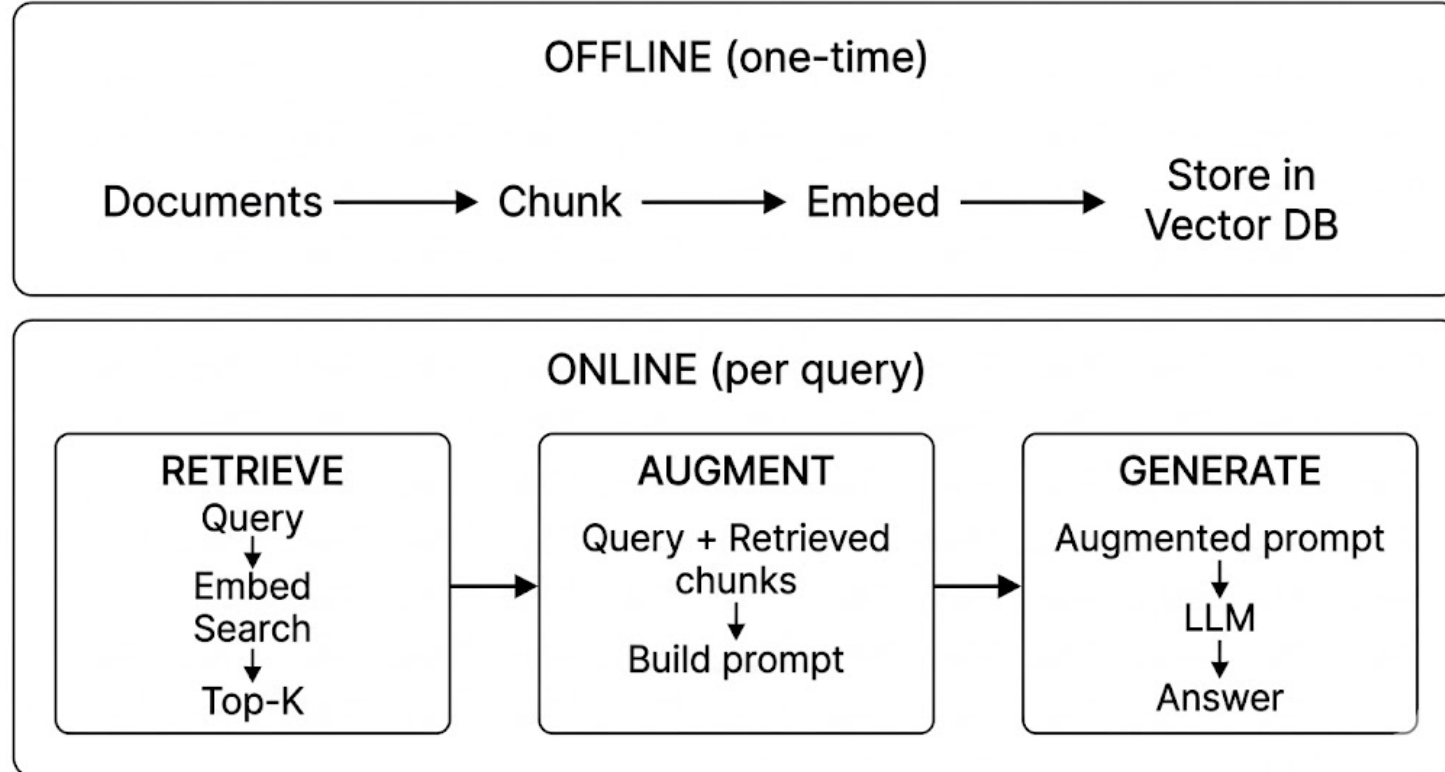
Why this separation matters:

- **Each component can be improved independently**
Swap a better embedding model into the retriever without touching the LLM. Upgrade the LLM without re-indexing your documents.
- **Failures are diagnosable**
If the answer is wrong but the retrieved passages are correct, the problem is in generation (prompt, model capability). If the retrieved passages are irrelevant, the problem is in retrieval (embeddings, chunking).
- **Costs are isolated**
Retrieval (vector search) is cheap and fast. Generation (LLM inference) is expensive. You can optimise each independently.

RAG INFERENCE PIPELINE

Just retrieval of the relevant documents is not enough; the generated answer is the desired output

Retrieval is achieved through searching, and it augments the generation process



This forms the complete RAG pipeline:
"Retrieval-Augmented" Generation

RAG INFERENCE PIPELINE

RAG systems follow a three-stage inference flow:

1. Stage 1 - Retrieve

- The user query is embedded using the same model that was used to embed the documents
- The vector store returns the top-K most similar chunks based on a distance metric (cosine, L2, or inner product)

2. Stage 2 - Augment

- The retrieved chunks are injected into the LLM's prompt alongside the original query
- This "augmented prompt" gives the model access to external knowledge it never saw during training
- The augmentation step also includes prompt engineering that tells the model to base its answer on the provided context (and covering cases when the context is insufficient/query is out of scope)

3. Stage 3 - Generate

- The LLM processes the augmented prompt and generates a response
- Because the relevant evidence is right there in the context window, the model can produce grounded, accurate, and citable answers

SEMANTIC SEARCH LIMITATIONS

- The retrieval stage uses vector similarity search
- It improves over keyword search but has several systematic limitations, especially in retrieval pipelines
- Embedding models capture semantic similarity, but they are not perfect representations of meaning. Some of the limitations are:
 - Loss of exact keyword matching
 - Difficulty with rare or technical terms
 - Sensitivity to chunking
 - Difficulty with numerical or structured data
 - Limited explainability
 - Embedding model bias and knowledge limits
 - Semantic over-generalisation of loosely related concepts
 - Difficulty with multi-intent queries

Let's see a few other limitations which are more relevant in context of RAG

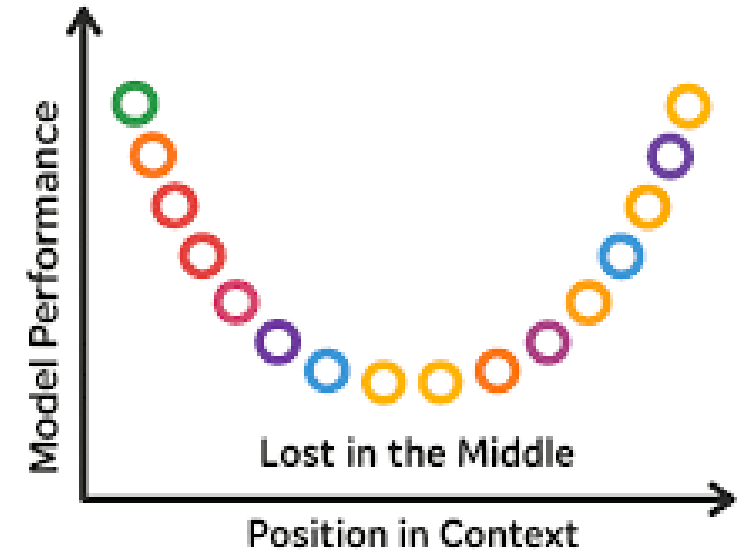
SEMANTIC GAP

Embedding models capture semantic similarity, but they are not perfect representations of meaning. They can:

- Confuse topically similar but factually different passages
A document about dog diseases and another about cat diseases may both appear similar because they relate to animal health
- Miss domain-specific jargon if the embedding model was not trained extensively on that terminology
- Struggle with negation and subtle phrasing
Sentences such as “the drug is effective” and “the drug is not effective” can produce very similar embeddings because most tokens overlap

LOST IN THE MIDDLE

- When many chunks are retrieved and inserted into the prompt, LLMs tend to pay more attention to the beginning and end of the context window and less to information placed in the middle
- This behaviour is known as the “lost in the middle” problem (Liu et al., 2023)
- As a result:
 - Retrieving too many chunks can reduce answer quality
 - Often 5–8 highly relevant chunks perform better than 20 loosely related ones



*In other words, generated answer accuracy is typically **highest when the relevant information appears near the beginning or the end of the context, and lowest when it appears in the middle***

RELEVANCE $\neq$ USEFULNESS

- Embedding similarity measures topical closeness, not whether a passage directly answers the question
- Example:
 - Query: *"Python list comprehension syntax"*
 - Retrieved chunk: *"Python list methods such as `append()`, `extend()`, and `pop()`"*
- The chunk is semantically related, but it does not contain the specific information needed to answer the question

CONTEXT CONTINUITY PROBLEMS

- Retrievers operate on individual chunks independently
- They **lack cross-chunk reasoning** and cannot combine information across multiple passages
- Example: For a corpus having documents related to multiple policies, identifying specific clauses related to a particular policy might be hard
- Example:
 - Chunk A: *The policy states employees must submit leave requests through the HR portal*
 - Chunk B: *The HR portal requires manager approval before submission*
 - Answering a question about the *full leave request process* requires combining both pieces of information

- On the other hand, when chunks contain multiple topics or excessive surrounding text, the embedding may represent the broader theme rather than the specific detail needed
- This is known as **context dilution**
- Example:
 - The employee handbook describes *company culture, leave policies, remote work rules, and travel reimbursements*
 - The query asks for *travel reimbursement rules*
 - Because the chunk covers many topics, the relevant information may be semantically diluted, making retrieval less precise

TOP-K RETRIEVAL

The simplest retrieval strategy:

Embed the query, compute similarity against all stored vectors, return the K most similar chunks

Query: *"What are the side effects of aspirin?"*

Top-K (k=3) retrieval:

1. Rank 1 (sim: 0.92)

"Common side effects of aspirin include irritation, bleeding, and allergic reactions..."

2. Rank 2 (sim: 0.87):

"Aspirin belongs to the NSAID class and works by inhibiting COX enzymes..."

3. Rank 3 (sim: 0.81):

"Ibuprofen side effects are similar to aspirin, including GI issues..."

Choosing K :

- Too small ($K = 1-2$):
Risk missing relevant context. If the best chunk is slightly off-topic, there's no backup.
- Too large ($K = 15-20$):
Floods the LLM's context window with marginally relevant text. Increases cost, latency, and "lost in the middle" effects.
- Typically, $K = 3-5$ for focused questions, $K=5-10$ for broad or multi-faceted questions

We can tune K using offline evaluations or A/B tests based on accuracy, factuality, and grounding

FILTERING

- Embeddings may retrieve topically similar but incorrect chunks
- Filtering is the process of removing irrelevant documents after (or before) retrieval to improve the quality of context passed to the LLM
- Its purpose is to reduce noise and enforce constraints so that only the most relevant, valid, and safe context reaches the LLM
- Filtering can be used in different places in the pipeline:
 - **Pre-Filtering (Before Retrieval):** Applied on metadata, reduces search space for a faster retrieval
 - **Post-Filtering (After Retrieval):** Applied on retrieved results, improves precision for a cleaner context
- There are several types of filtering techniques for RAG:
 - **Metadata Filtering:** Based on attributes (source, domain, date)
 - **Score Thresholding:** Remove low similarity results
 - **Keyword / Boolean Filtering:** Enforce lexical constraints
 - **Deduplication:** Remove repeated or overlapping chunks
 - **Access Control Filtering:** Restrict unauthorised data

METADATA FILTERING

Pure vector similarity ignores structured attributes of documents

Metadata filtering lets you constrain the search space before or during the vector search:

```
Query: "Latest compliance requirements"

Without filtering:
  → Might return documents from 2019, 2021, and 2024

With metadata filter (year >= 2024):
  → Only returns documents from 2024 and later
```

Metadata filtering is especially powerful when combined with semantic search:
narrow the candidate pool with structured filters, then rank within that pool by vector similarity

We saw an example of pre-retrieval metadata filtering while adding query conditions in vector databases

METADATA FILTERING

Common metadata fields:

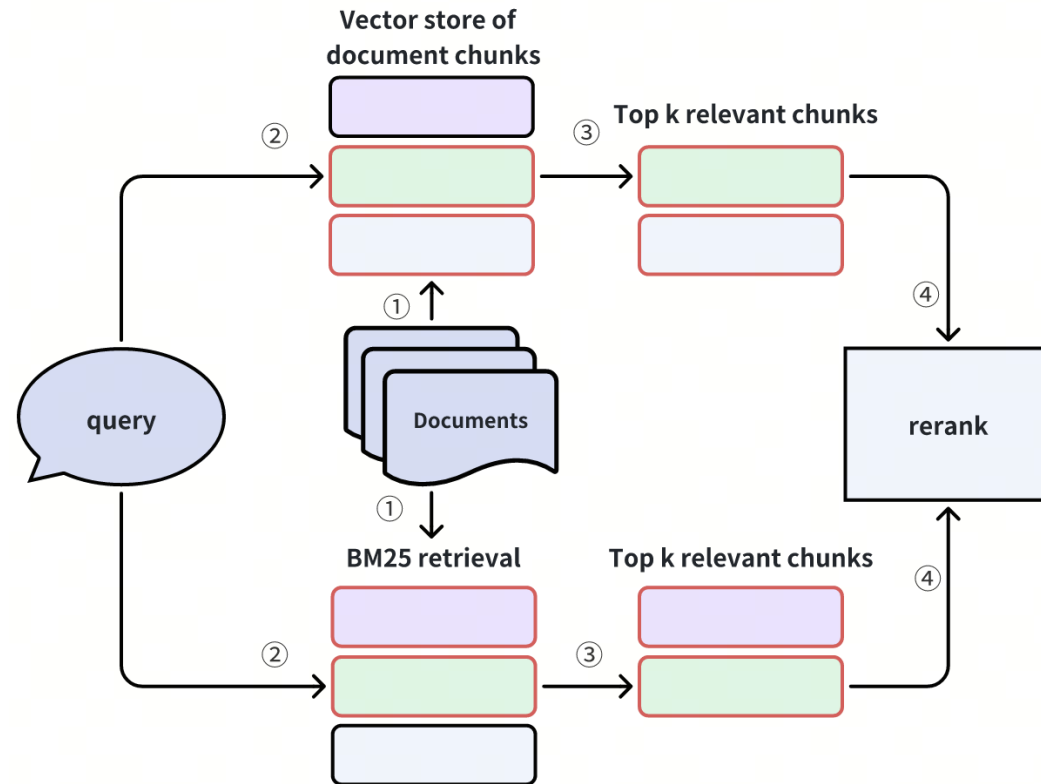
Field	Use Case
<i>source</i>	Filter by document origin (e.g., only internal docs, not web crawls)
<i>date / year</i>	Recency filtering
<i>department</i>	Access control (only show HR docs to HR)
<i>document type</i>	Filter by format (policy, FAQ, technical spec)
<i>language</i>	Multilingual corpora
<i>chunk_index</i>	Retrieve adjacent chunks for context expansion

A problem with this is the "**Empty Result Set**" problem, where highly relevant semantic matches are discarded because they fail strict filter criteria, leading to missing context

Metadata filtering creates a trade-off: **post-filtering** risks returning zero results, while **pre-filtering** can cause a **performance collapse** by turning fast vector searches into slow scans

HYBRID RETRIEVAL

Hybrid search combines dense retrieval (vector similarity) with sparse retrieval (keyword matching like BM25) and merges their results into a single ranked list



Hybrid search is useful because no single method covers all cases well

SCORE FUSION

We combine the scores from dense and sparse retrievals into a single ranking. The difficulty is that these scores may have different scales and distributions. We can use a few score fusion methods like

Linear Score Fusion

This is a weighted sum of the two scores:

$$S(d) = \alpha \cdot S_{\text{sparse}}(d) + (1 - \alpha) \cdot S_{\text{dense}}(d)$$

- The weight $\alpha \in [0,1]$ can be tuned
- The doc scores need to be normalised here (e.g., min-max, Softmax)
- It is simple but sensitive to score calibration

Reciprocal Rank Fusion (RRF)

$$S(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

- The constant k is usually taken as 60
- This does not use raw scores, but ranks of documents
- It is very robust and doesn't need normalisation

HYBRID SEARCH USE CASES

Hybrid search is not always necessary

Situation	Recommendation
Queries are natural language questions about concepts	Dense is usually sufficient
Queries contain exact entities, IDs, codes, error messages	Hybrid (sparse catches exact matches)
Corpus is highly technical with domain jargon	Hybrid (sparse handles rare terms)
You need maximum recall (legal, compliance)	Hybrid (covers both signal types)
Prototype / learning	Start with dense, add sparse later
Production with diverse query types	Hybrid is the safe default

Most modern vector databases support hybrid search natively:

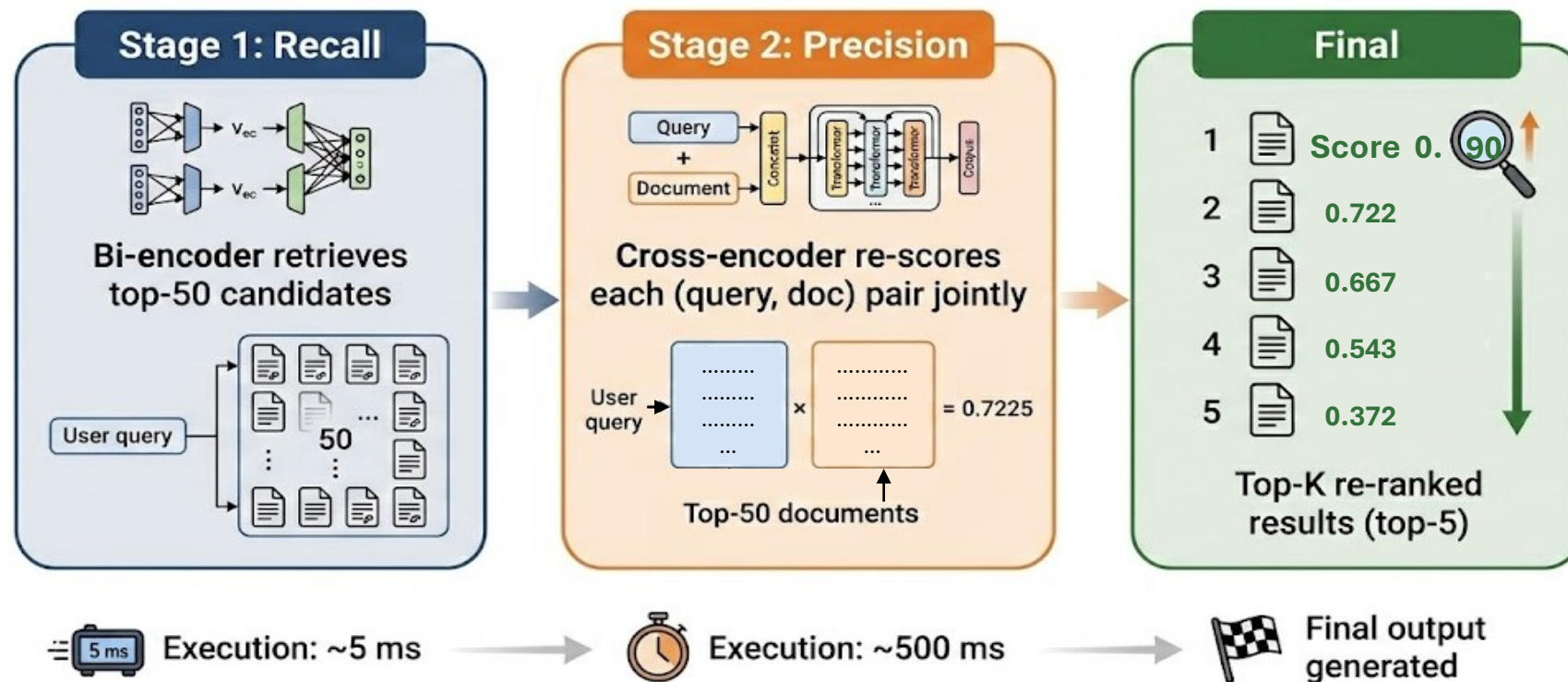
- **Qdrant:** Supports sparse vectors alongside dense vectors in the same collection
- **Weaviate:** Built-in hybrid search with configurable alpha (α)
- **Pinecone:** Supports sparse-dense vectors in a single index
- **Elasticsearch:** Combines kNN vector search with BM25 in a single query
- **ChromaDB + BM25:** ChromaDB is dense-only, but you add sparse search and fuse results yourself

RE-RANKING

Basic top-K retrieval uses a **bi-encoder**: query and documents are embedded independently
This is fast but coarse and can return unusable or incorrectly ordered chunks

Thus, after filtering, we add another layer for re-ranking the results

Re-ranking adds a second, more accurate stage (usually using a **cross-encoder**)



RE-RANKING TYPES

○ Cross-Encoder Re-ranking:

- Takes the (*query*, *document*) pair jointly as input; outputs a relevance score
- Provides high accuracy, but is computationally expensive; not scalable for large corpus

○ Bi-Encoder Re-ranking:

- Same architecture as retrievers but fine-tuned for ranking
- Faster but less precise than cross-encoders; these can be used in latency-sensitive systems

○ Late Interaction Models:

- Hybrid between bi-encoder and cross-encoder; considers token-level similarity instead of full interaction
- More expressive than bi-encoder, cheaper than cross-encoder
- For example, ColBERT (though this resembles a retriever more closely than a re-ranker)

○ LLM-based Re-ranking:

- Using LLMs to judge relevance; framed as scoring, ranking, pairwise-comparison
- Handles complex queries with better reasoning, but this is costly to use with high latency

RE-RANKING FORMULATIONS

- Re-ranking can be done based on various scoring methods:
 - **Pointwise:** Score each document independently
 - **Pairwise:** Compare two documents
 - **Listwise:** Rank entire set jointly (most optimal but complex)
- Common loss functions that can be used as training objectives for re-ranking are:
 - Cross-entropy (binary relevance)
 - Margin ranking loss
 - Contrastive loss (for embedding-based models)
- Re-ranking is significantly useful when queries are ambiguous or nuanced, and the retrieved documents are semantically close but not exact (some domains (legal, medicine) require precision)
- We can also use hybrid scoring (combine hybrid retrieval + re-ranking), query expansion, or multi-stage re-ranking to further improve the results

CROSS-ENCODER RE-RANKING

- A cross-encoder processes the query and candidate passage together as a single input, allowing it to model fine-grained interactions between query tokens and document tokens
- A bi-encoder cannot do this because it embeds them independently
- Cross-encoders use transformer architecture (e.g., a BERT-style architecture is common for cross-encoder re-ranking models)
- When to use re-ranking:
 - When retrieval precision is critical (medical, legal, financial)
 - When you can afford the latency (~100-500ms for re-ranking 50 candidates)
 - When your bi-encoder retrieves "almost right" results that need refinement
- Popular re-ranking models:
 - cross-encoder/ms-marco-MiniLM-L-6-v2 : fast, good quality
 - BAAI/bge-reranker-large : state-of-the-art accuracy
 - Cohere Rerank API: managed service, no GPU required

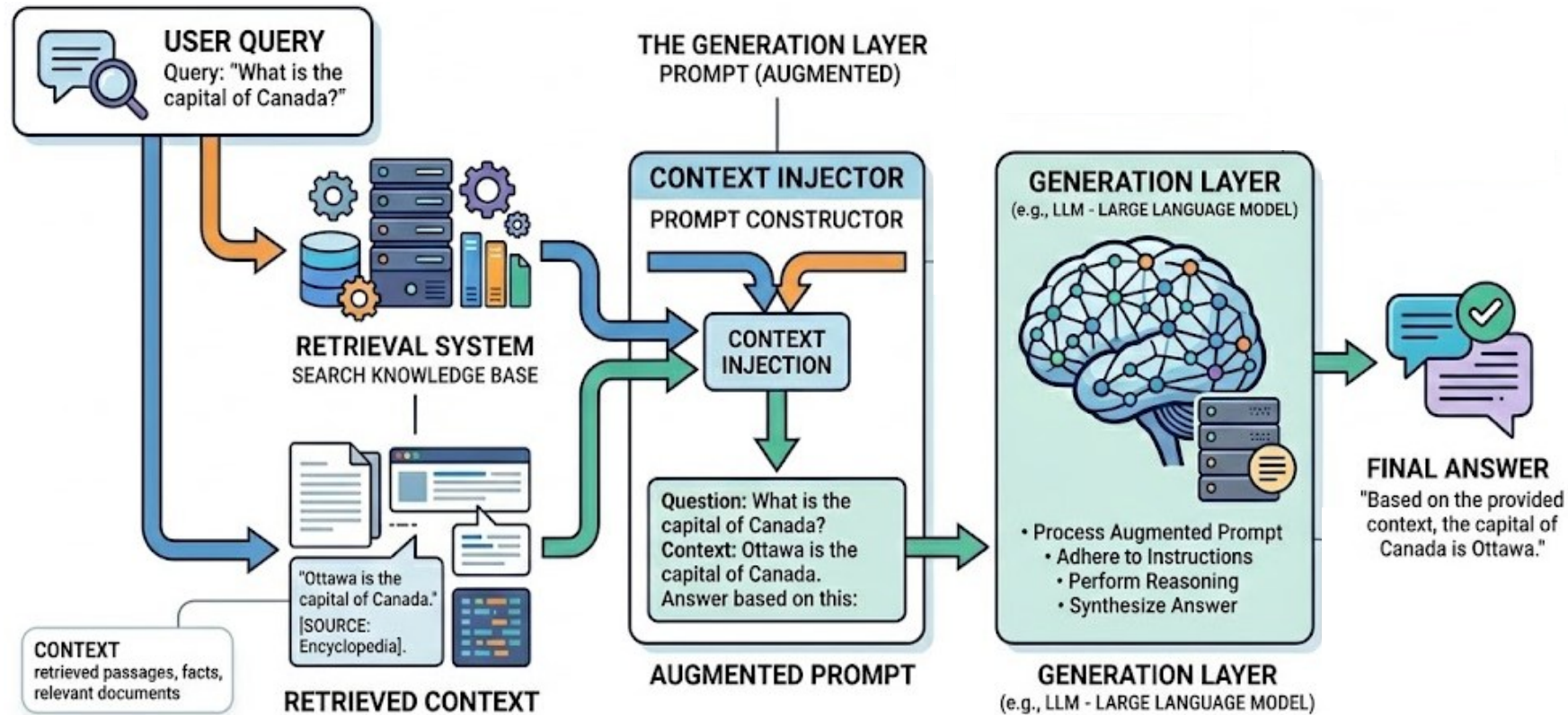
RETRIEVAL vs RE-RANKING

Metric	Bi-encoder Only (Base Retrieval)	Bi-encoder + Cross-encoder (Re-ranking)
Workflow	Direct Vector Search (ANN)	Retrieval → Re-rank top K candidates
Latency (K=50)	Low (~5–15ms)	Moderate to High (+50–200ms)
Relative Cost	\$ (Efficient; CPU-friendly)	\$\$\$ (GPU-intensive for inference)
Scalability	High (scales to millions of docs)	Medium (limited by K candidates)
Accuracy/MRR	Good (semantic match)	Superior (deep contextual scoring)

- When scaling search systems, adding a cross-encoder significantly improves relevance (accuracy) but introduces a "re-ranking tax" in terms of time and compute
- This is because they perform full-attention over the query and document pair together, rather than comparing pre-computed embeddings
- We can limit the Cross-encoder to only the top-K candidates (e.g., $K = 50$) to keep latency within acceptable bounds for user-facing applications
- Use Bi-encoders for broad recall and Cross-encoders for precision at the very top of the results list

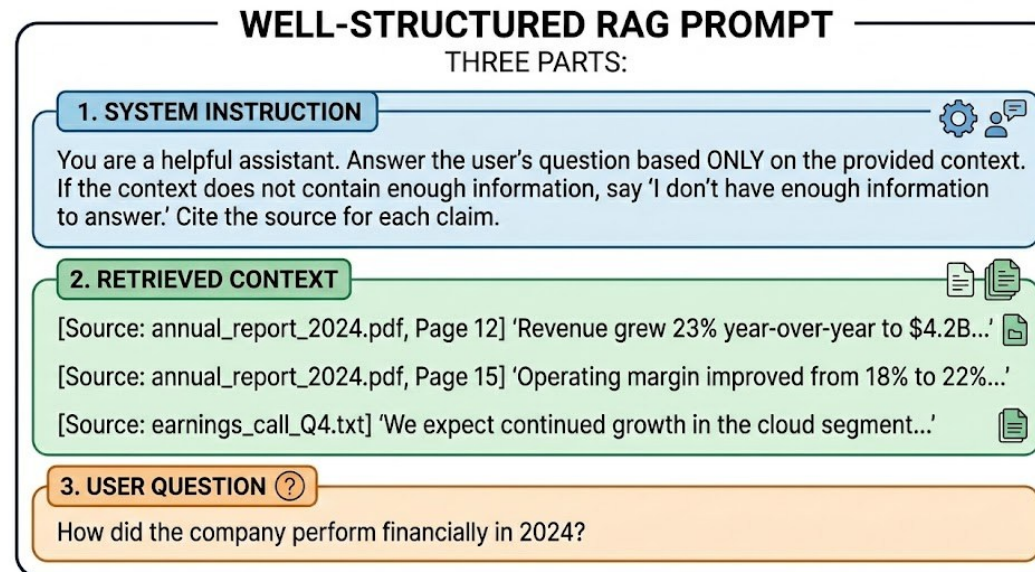
CONTEXT INJECTION

Once the retriever has found the relevant chunks, they must be injected into the LLM's prompt
This is the "**augment**" step: the bridge between retrieval and generation



CONTEXT INJECTION BEST PRACTICES

1. **Include source metadata** with each chunk (file name, page number, date). This enables the LLM to cite its sources and helps users verify answers.
2. **Order chunks by relevance**, most relevant first. Due to the "lost in the middle" problem, the LLM pays most attention to the beginning and end of the context.
3. **Use delimiters** (---, [Source: ...]) to separate chunks so the model can distinguish between different sources.
4. **Constrain the model**. The system instruction should explicitly tell the model to use only the provided context and to admit when it doesn't know. Manage context window budget wisely.



CONTEXT WINDOW BUDGET

Every LLM has a maximum context length. A RAG prompt must fit within it:

```
Total context window (e.g., 8,192 tokens)
├── System instruction:    ~100-200 tokens
├── Retrieved chunks:     ~2,000-6,000 tokens ← the bulk
├── User query:           ~20-100 tokens
└── Reserved for response: only ~500-2,000 tokens
```

Trade-off: Injecting more chunks provides more evidence but leaves less room for the model's response and increases latency and cost.

For example, consider this scenario of a system prompt for a model with context window of 32K tokens:

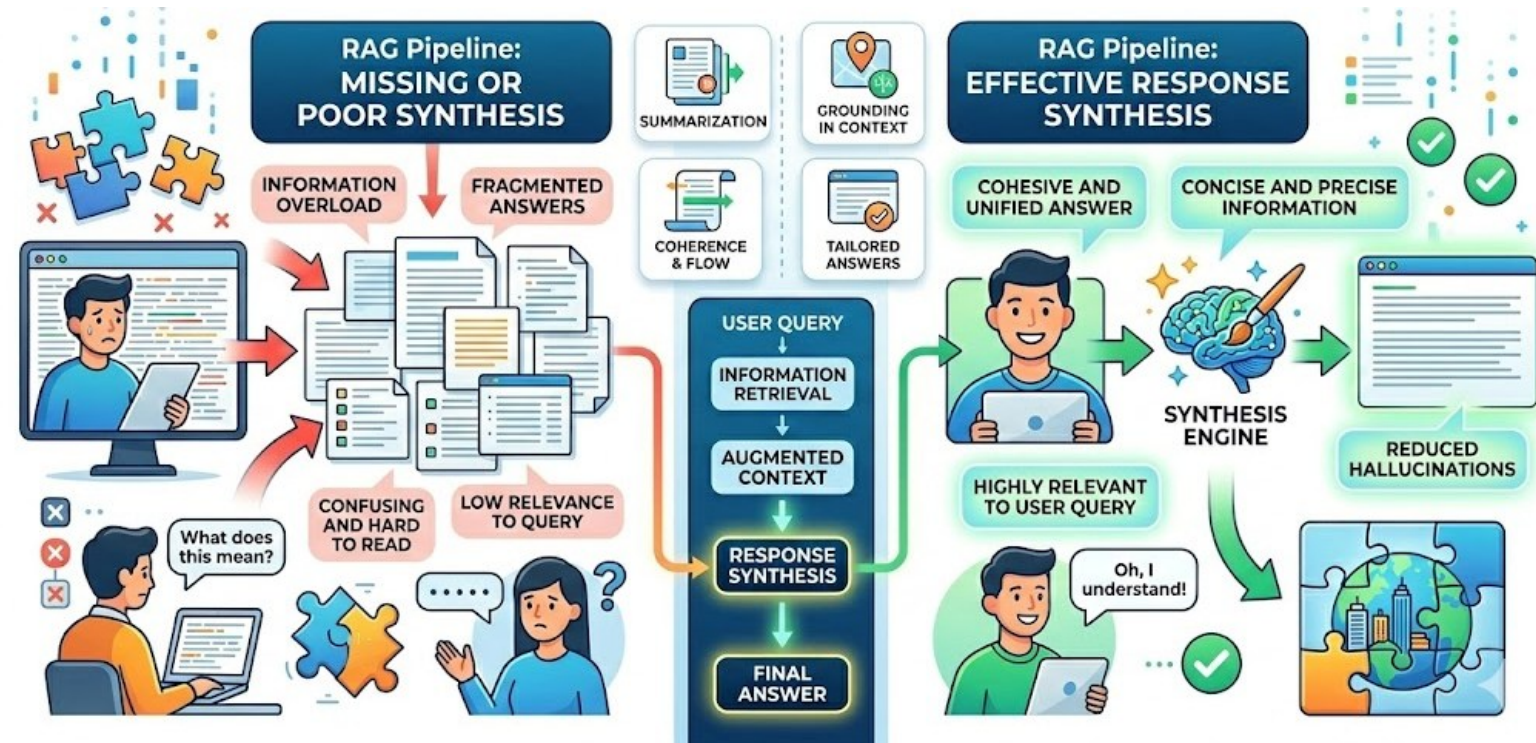
```
"You are a Senior DevOps Engineer. Based on these 25 manual entries [30,500tokens],
answer the following user question: My SSL handshake is failing on port 443, help
me debug. Provide a step-by-step resolution."
```

The system prompt itself will have an approximate token size of 31K

A common pattern is to retrieve more candidates (top-20), re-rank, and inject only the top-3-5 after re-ranking

RESPONSE SYNTHESIS STRATEGIES

- The generator does not simply copy-paste from the retrieved chunks, it synthesises a response
- Several synthesis strategies exist:
 - Stuffing (Simple Concatenation)
 - Map-Reduce
 - Refine (Iterative)
 - Tree Summarisation



STUFFING

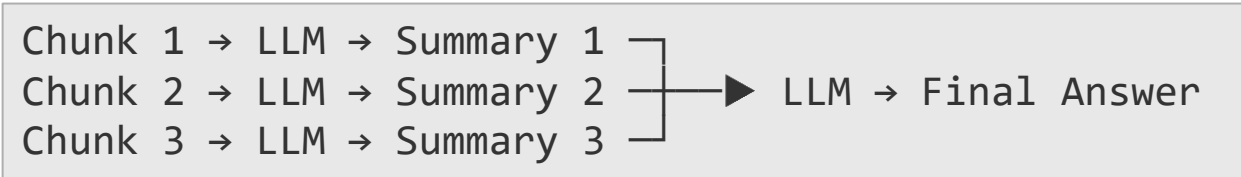
The simplest approach: stuff all retrieved chunks into a single prompt and ask the LLM to answer

```
prompt = f"""Based on the following context, answer the question.  
  
Context:  
{chr(10).join(chunks)}  
  
Question: {query}  
Answer: """
```

- **Pros:** Simple, single LLM call, low latency
- **Cons:** Limited by context window size; Fails if the total chunk text exceeds the limit
- **Best for:** Small numbers of short chunks (3-5 chunks)

MAP-REDUCE

When the retrieved content exceeds the context window, process chunks in parallel ("map"), then combine the partial answers ("reduce"):



- **Pros:** Handles unlimited amounts of retrieved content
- **Cons:** Multiple LLM calls (higher cost and latency); Intermediate summaries may lose detail
- **Best for:** Long-document QA, summarisation over large corpora

REFINING

Process chunks one at a time, iteratively refining the answer:

```
Chunk 1 → LLM → Answer v1  
Chunk 2 + Answer v1 → LLM → Answer v2  
Chunk 3 + Answer v2 → LLM → Answer v3 (final)
```

- **Pros:** Each iteration can add nuance and detail; Good for complex, multi-faceted questions
- **Cons:** Sequential (slow); Later chunks can overwrite earlier, important information
- **Best for:** Questions requiring synthesis across multiple sources with evolving detail

TREE SUMMARISATION

Recursively summarise pairs of chunks until a single answer remains:

```
Chunk 1 + Chunk 2 → LLM → Summary A
Chunk 3 + Chunk 4 → LLM → Summary B
Summary A + Summary B → LLM → Final Answer
```

- **Pros:** Balanced processing, parallelisable at each level
- **Cons:** Multiple LLM calls; may over-compress
- **Best for:** Large retrieval sets where map-reduce is too flat

To summarise,

Strategy	LLM Calls	Handles Large Context	Latency	Quality
Stuffing	1	No (limited by window)	Low	Good (if it fits)
Map-Reduce	$N + 1$	Yes	Medium	Good
Refining	N	Yes	High (sequential)	Very good
Tree	$\sim N$	Yes	Medium (parallel)	Good

GROUNDING AND HALLUCINATION CONTROL

Even with retrieved context, the LLM can still hallucinate. Several techniques help keep it grounded:

- **Explicit grounding instructions:**

- “Answer based ONLY on the provided context. Do not use your training knowledge.”
- Primarily, it can minimise hallucination due to temporal drift

- **Quote-based answering:**

- “Support each claim with a direct quote from the context.”
- This approach can resolve hallucination due to factual fabrication

- **Confidence calibration:**

- “If you are not sure, say ‘Based on the provided documents, I cannot confidently answer this.’ ”
- It resolves over-confidence in hallucinated responses

- **Source citation format:**

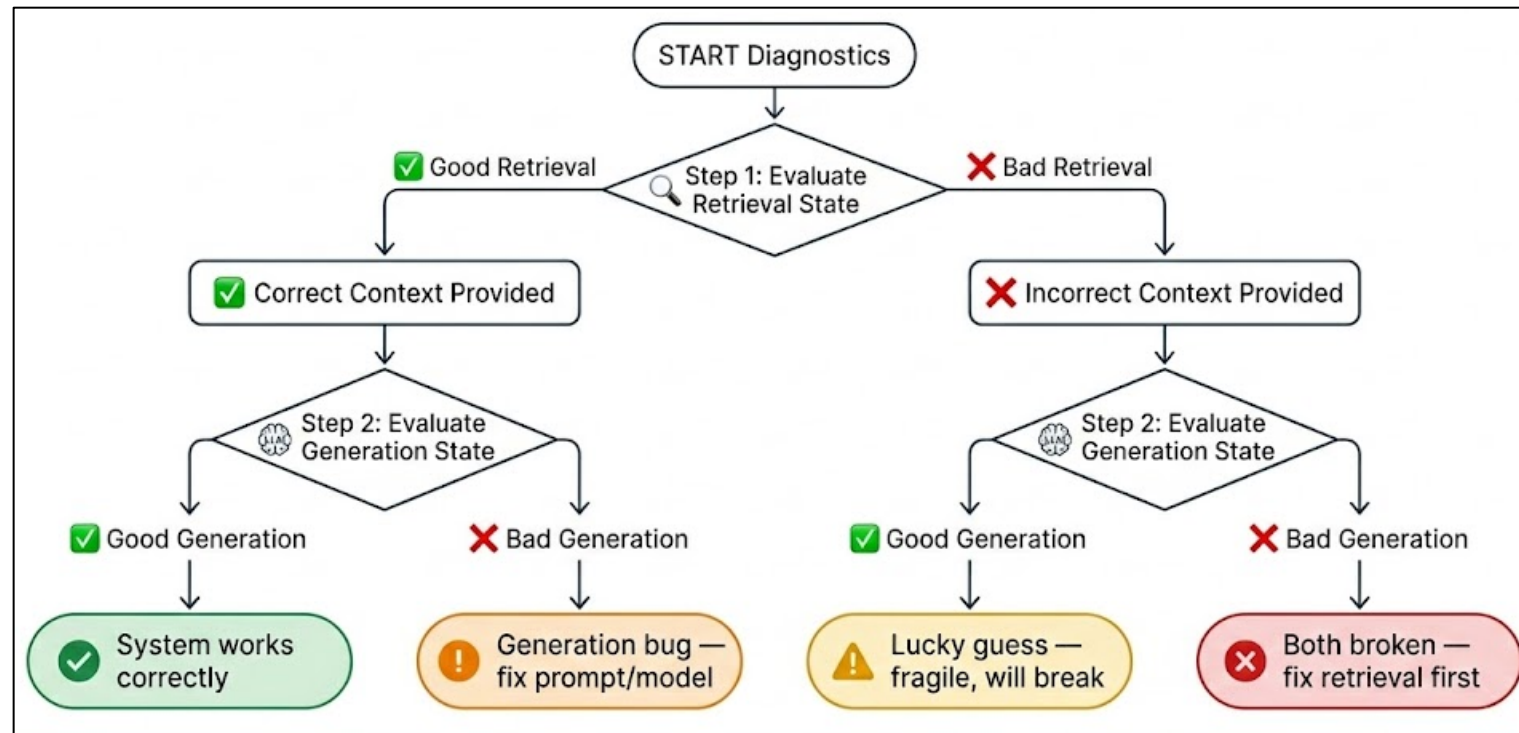
- Require the model to output [Source: filename, Page X] after each claim
- This makes fabrication easy to spot and can resolve hallucination due to factual fabrication

- **Post-generation verification:**

- A secondary LLM call checks whether each claim in the response is supported by the retrieved context
- It is effective for all the hallucination type

WHY RAG EVALUATION IS HARD

- A RAG system has two components that can fail independently: the retriever and the generator
- A wrong answer can be caused by poor retrieval (right question, wrong context), poor generation (right context, wrong answer), or both
- Evaluation must disentangle these failure modes



RETRIEVAL METRICS

These measure whether the retriever is surfacing the right documents

Metric	What It Measures	How to Compute
Hit Rate (Recall@K)	Did the correct document appear anywhere in the top-K?	$\frac{\text{queries with at least 1 relevant doc in top } K}{\text{total queries}}$
Mean Reciprocal Rank (MRR)	How high does the first relevant document rank?	Average of $\left(\frac{1}{\text{rank of first relevant doc}}\right)$ across queries
Precision@K	What fraction of top-K results are relevant?	$\frac{\text{relevant in top } K}{K}$
NDCG@K	Are relevant documents ranked higher than irrelevant ones?	Normalised Discounted Cumulative Gain
Context Relevance	Are the retrieved chunks actually useful for answering the query?	LLM-as-judge scores each chunk's relevance (used by RAGAS)

GENERATION METRICS

These measure whether the LLM's response is correct and grounded

Metric	What It Measures	How to Compute
Faithfulness	Is the answer supported by the retrieved context? (No hallucination)	LLM-as-judge checks if each claim can be attributed to a context chunk
Answer Relevance	Does the answer actually address the question?	LLM-as-judge or embedding similarity between answer and question
Answer Correctness	Is the answer factually correct?	Compared against ground-truth answers (exact match, F1, or LLM-judge)
Answer Completeness	Does the answer cover all aspects of the question?	LLM-as-judge with a rubric

SYSTEM-LEVEL METRICS

These measure the operational health of the RAG system

Metric	What It Measures	Target
Latency (E2E)	Time from query to response	< 2-3s for interactive; < 10s for complex
Retrieval latency	Time for vector search + re-ranking	< 100-200ms
Generation latency	Time for LLM inference	Depends on model and output length
Cost per query	Embedding + retrieval + LLM tokens	Varies; track and optimise
Throughput	Queries per second the system can handle	Depends on scaling requirements

EVALUATION FRAMEWORKS

Several frameworks can be used automate RAG evaluation

Metric	What It Measures	Key Metrics
RAGAS	LLM-as-judge for component-level evaluation	Faithfulness, Answer Relevance, Context Precision, Context Recall
DeepEval	Unit-test style assertions for LLM outputs	Hallucination, Answer Relevancy, Contextual Relevancy, Bias, Toxicity
TruLens	Feedback functions with instrumented pipelines	Groundedness, Relevance, Sentiment
LangSmith	End-to-end tracing and evaluation for LangChain apps	Custom evaluators, tracing, datasets

However, these are just starting points
Enterprise solutions often require a custom framework

OBSERVABILITY SIGNALS

Beyond batch evaluation, production RAG systems need continuous observability, real-time signals that tell you when something is going wrong:

Signal	What It Reveals	Action If Abnormal
Retrieval hit rate	% of queries where at least 1 relevant chunk is retrieved	Low? Improve embeddings, chunking, or add query expansion
Average similarity score	How close are top-K results to the query?	Declining? Embedding drift or new query patterns not covered by the corpus
Empty retrieval rate	% of queries that return no results above threshold	High? Lower threshold, expand query, or add documents
Chunk utilisation	How many of the retrieved chunks actually appear in the LLM's answer?	Low? You are over-retrieving; reduce K or compress

BUILDING AN EVALUATION DATASET

- Automated metrics are only as good as the evaluation dataset. Build yours by:
 - **Annotate ground-truth answers:** what is the correct answer for each query?
 - **Annotate relevant documents:** which documents should be retrieved for each query?
 - **Include adversarial cases:** queries that should return "I don't know", queries with ambiguous answers, queries that require multiple chunks
 - **Version and update:** the eval set should evolve as your corpus and user base grow

A good starting size is 50-100 annotated query-answer pairs for initial development, growing to 500+ for production systems